InfraWatch

Infrastructure Inventory Management

User Manual & Administration Guide

Version 0.1.0 | March 2026

For IT Operations, DevOps, and Infrastructure Teams

Table of Contents

1. Introduction
2. Installation & Deployment
3. Initial Setup & Configuration
4. Dashboard Overview
5. Host Inventory Management
6. Alert Management & Remediation
7. Change Tracking
8. End-of-Life (EOL) Tracking
9. Host Groups
10. Dependency Mapping
11. Compliance Scoring
12. Network Discovery
13. Scan Target Management
14. Scheduled Reports
15. Notifications
16. API Reference
17. Updating & Maintenance
18. Troubleshooting

1. Introduction

InfraWatch is an agentless infrastructure inventory management platform that automatically discovers, scans, and monitors your servers, containers, and cloud resources. It provides real-time visibility into installed packages, running services, security vulnerabilities, and compliance posture across your entire fleet.

Key Features

- **Agentless Scanning** - No software to install on target hosts. Connects via SSH, WinRM, Kubernetes API, AWS API, VMware vSphere, or Docker daemon.
- **Package & Vulnerability Tracking** - Discovers installed packages, checks for newer versions, and creates alerts for outdated or vulnerable software.
- **End-of-Life Monitoring** - Tracks products approaching or past their end-of-life dates to prevent security gaps.
- **Compliance Scoring** - Rates each host 0-100 across five weighted factors, with fleet-wide and group-level aggregation.
- **Dependency Mapping** - Visualizes network connections between hosts to understand service dependencies and blast radius.
- **Change Auditing** - Records every host, package, service, and configuration change for audit trails.
- **Automated Remediation** - Generates platform-aware fix commands (apt, yum, pip, npm, etc.) for each alert.
- **Scheduled Reports** - Generates and emails PDF reports on a configurable schedule.
- **Multi-Channel Notifications** - Sends alerts via Slack, Microsoft Teams, email, or generic webhooks.

Architecture

InfraWatch consists of three components deployed as Docker containers:

- **API Server** (Node.js/Express) - REST API, background services (scan orchestration, version checking, compliance scoring, notifications), and database access.
- **Web Frontend** (React/Vite) - Single-page application served by nginx in production. Communicates exclusively through the API.
- **PostgreSQL Database** - Stores all host inventory, scan results, alerts, compliance scores, and configuration. Credentials are encrypted at rest with AES-256-GCM.

Supported Scanner Types

Scanner	Protocol	Discovers
SSH (Linux)	SSH	Packages (apt/yum/apk), services, OS, network
WinRM	WinRM	Windows programs, services, OS info
Kubernetes	K8s API	Container images, pods, namespaces
AWS	AWS API	EC2, RDS, Lambda instances and packages
VMware	vSphere API	Virtual machines, guest OS, services
Docker	Docker API	Running containers, images, ports
Network Discovery	ARP/ICMP/nmap	Live hosts, open ports, MAC addresses

2. Installation & Deployment

Prerequisites

- **Docker** (v20.10 or later) and **Docker Compose v2**
- **OpenSSL** (for generating secrets during setup)
- At least **2 GB RAM** and **10 GB disk space** for the application and database
- Network access to target hosts on their respective management ports (SSH/22, WinRM/5985, etc.)

Quick Start (Automated Setup)

The included setup script handles secret generation, configuration, building, and starting all services:

```
git clone <repository-url> infrawatch
cd infrawatch
./setup.sh
```

The setup script performs the following steps automatically:

- Verifies Docker, Docker Compose v2, and OpenSSL are installed
- Generates cryptographically random secrets: `DB_PASSWORD`, `MASTER_KEY`, and `API_KEY`
- Creates the `.env` file from the production template
- Builds all Docker images (multi-stage builds)
- Starts PostgreSQL, API server, and web frontend
- Waits for health checks to pass (up to 120 seconds)
- Reports the access URL when ready

Manual Deployment

If you prefer manual control over the deployment:

Step 1: Create the environment file

```
cp .env.production.example .env
```

Edit `.env` and set these required values:

Variable	Description	Default
`DB_PASSWORD`	PostgreSQL password	*required*

`MASTER_KEY`	AES-256 encryption key for credentials (32+ chars)	*required*
`API_KEY`	API authentication key	*required*
`DB_HOST`	PostgreSQL hostname	`postgres`
`DB_PORT`	PostgreSQL port	`5432`
`DB_NAME`	Database name	`infrawatch`
`DB_USER`	Database user	`infrawatch`
`PORT`	API server port	`3001`
`CORS_ORIGIN`	Allowed origin for web UI	`http://localhost`

Step 2: Build and start

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

Step 3: Verify health

```
curl http://localhost:3001/api/v1/health
```

Development Mode

For local development with hot-reload:

```
# Start PostgreSQL
docker compose up postgres -d

# Build the scanner package (dependency of server)
npm run build -w packages/scanner

# Start API server with hot-reload (terminal 1)
npm run dev -w packages/server

# Start web frontend with Vite HMR (terminal 2)
npm run dev -w packages/web
```

Note: In development mode, the API runs on port 3001 and the web UI on port 5173. Vite automatically proxies `/api` requests to the API server.

3. Initial Setup & Configuration

Environment Variables

All configuration is via environment variables in the `.env` file:

Email / SMTP Settings (optional)

Variable	Description	Default
`SMTP_HOST`	SMTP server address	*required*
`SMTP_PORT`	SMTP port	`587`
`SMTP_USER`	Email account username	*required*
`SMTP_PASS`	Email account password	*required*
`ALERT_EMAIL`	Recipient for alert digests	*required*

Tuning Parameters

Variable	Description	Default
`DB_POOL_MAX`	Max database connections	`20`
`VERSION_CHECK_INTERVAL_HOURS`	Package update check interval	`12`
`ALERT_DIGEST_HOUR`	Hour to send daily digest (0-23)	`8`
`REPORT_STORAGE_PATH`	Where generated reports are saved	`./data/reports`

Database Migrations

Migrations run automatically on server startup. For manual control:

```
# Apply pending migrations
npm run db:migrate -w packages/server

# Rollback last migration
npm run db:migrate:down -w packages/server

# Create a new migration
npm run db:migrate:create -w packages/server
```

API Authentication

All API endpoints (except `/health`) require the `X-API-Key` header when `API_KEY` is set in the environment. The web frontend automatically includes this header from `VITE_API_KEY` (baked in at build time).

```
curl -H 'X-API-Key: your-api-key' http://localhost:3001/api/v1/hosts
```

4. Dashboard Overview

The dashboard provides an at-a-glance view of your infrastructure health. It is the default landing page after login.

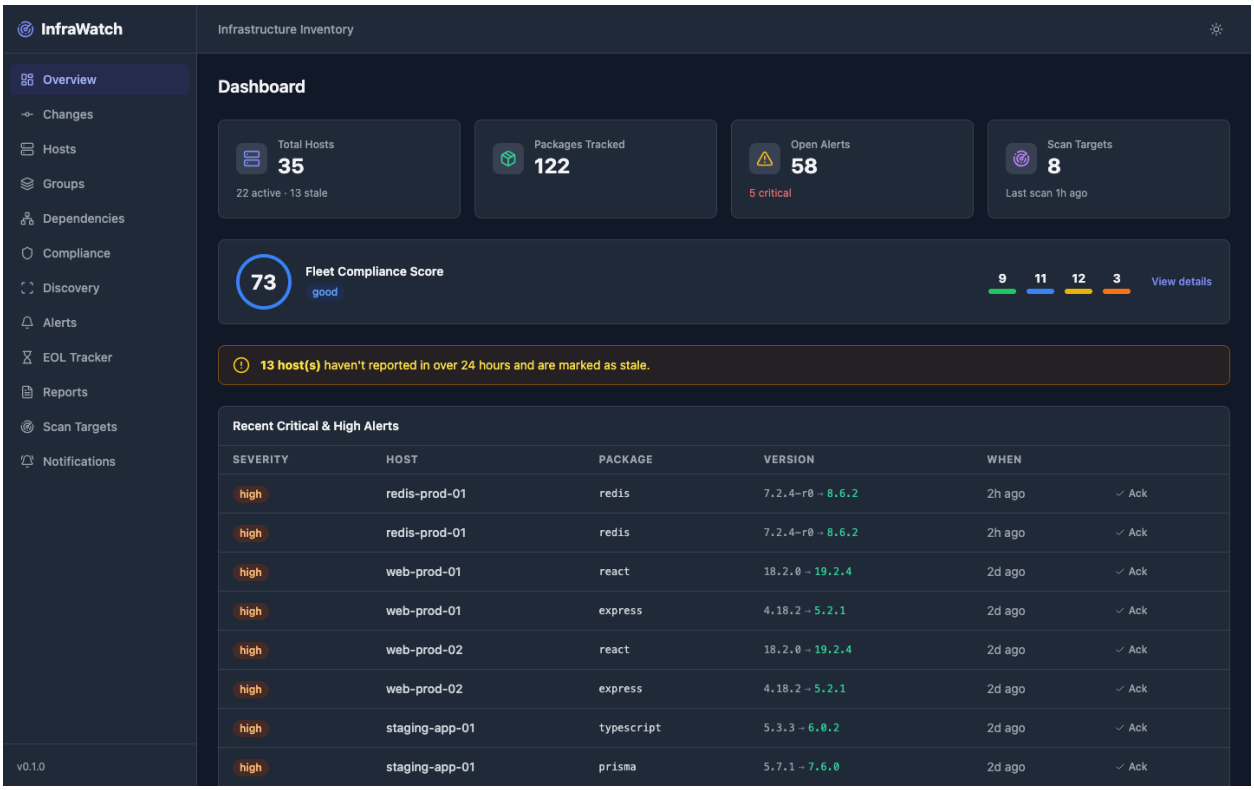

Figure 1: Dashboard showing fleet statistics, compliance score, recent alerts, groups, and changes

Dashboard Widgets

- **Total Hosts** - Count of all hosts with active/stale breakdown
- **Packages Tracked** - Total number of installed packages across all hosts
- **Open Alerts** - Unacknowledged alerts with critical count highlighted
- **Scan Targets** - Configured scan targets with time since last scan
- **Fleet Compliance Score** - Aggregate 0-100 score with classification badge and host distribution
- **Stale Hosts Warning** - Banner showing hosts that haven't reported in 24+ hours
- **Recent Critical & High Alerts** - Table of the most recent high-severity alerts with one-click acknowledge
- **Groups** - Cards for each host group showing member count and open alerts

- **Recent Changes** - Feed of the latest infrastructure changes (host discoveries, package updates, etc.)

5. Host Inventory Management

The Hosts page is the central inventory view of all discovered infrastructure. Hosts are automatically added when scan targets are configured and scanned.

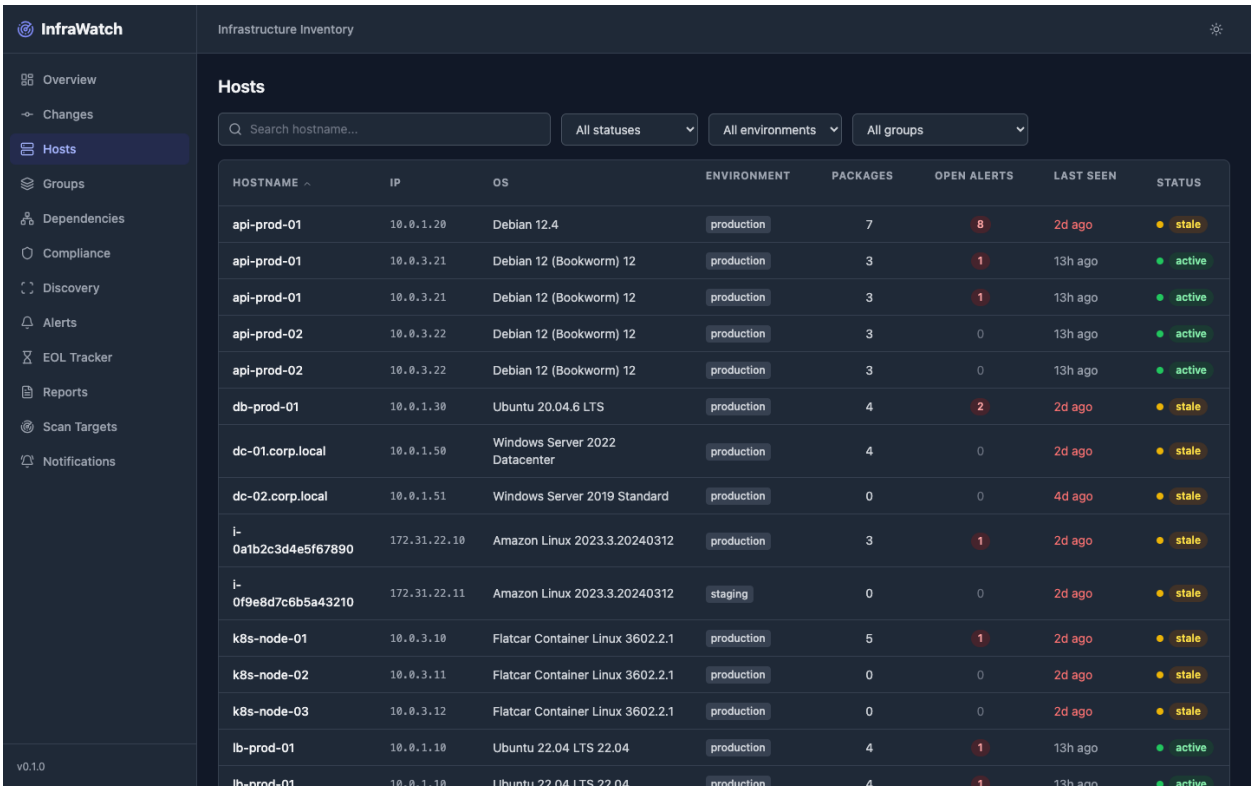

Figure 2: Host inventory with filtering by status, environment, and group

Host Table Columns

- **Hostname** - Clickable link to host detail page
- **IP** - Primary IP address
- **OS** - Operating system and version
- **Environment** - Tagged environment (production, staging, etc.)
- **Packages** - Number of installed packages
- **Open Alerts** - Count of unacknowledged alerts (color-coded by severity)
- **Last Seen** - Time since last successful scan
- **Status** - Active (green), Stale (yellow, 24h+ no report), or Decommissioned (red, 30d+ no report)

Filtering & Search

Use the filter bar at the top to narrow results by hostname (text search), status, environment, or group. Click column headers to sort. The table supports pagination for large inventories.

Host Detail View

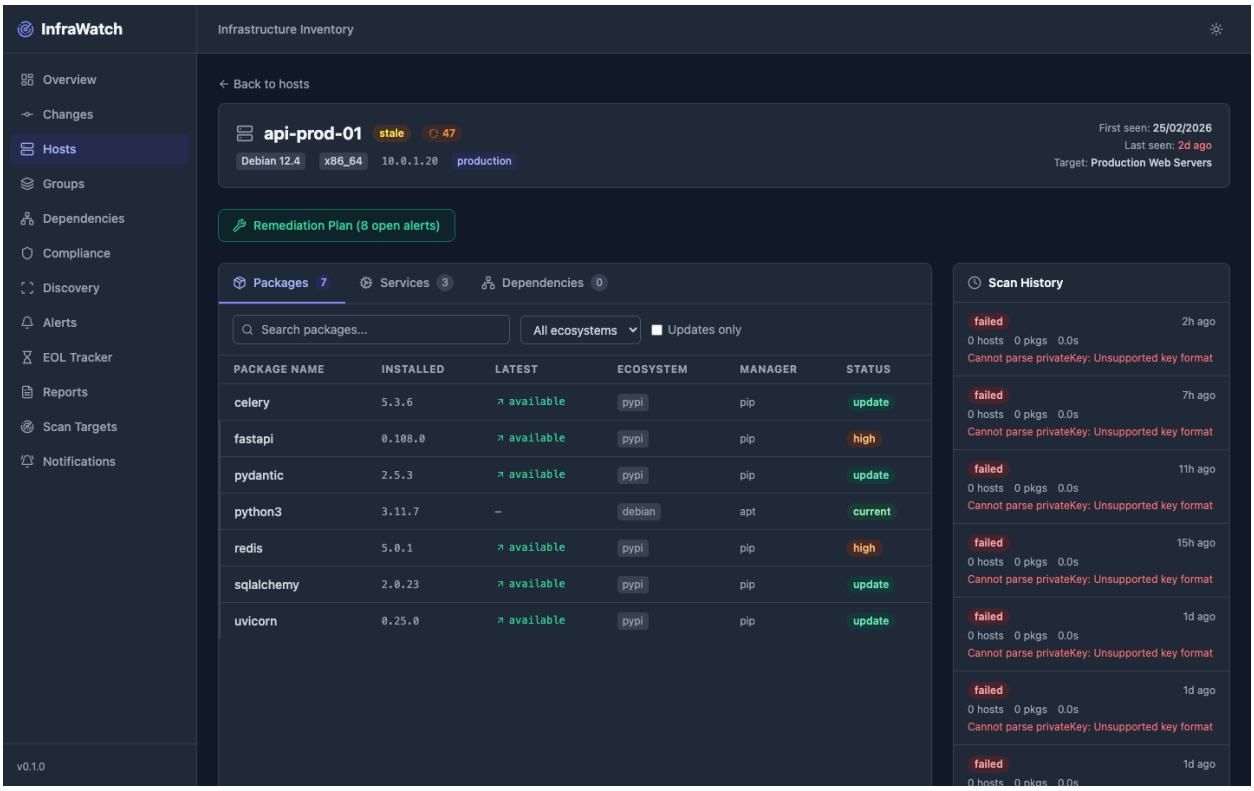

Figure 3: Host detail page showing packages, services, compliance badge, and remediation

Click any host row to view its full details. The host detail page includes:

- **Host header** - Hostname, status badge, compliance score badge, OS info, IP, environment tag
- **Remediation Plan button** - Opens panel with fix commands for all open alerts on this host
- **Packages tab** - Installed packages with installed/latest version comparison and ecosystem filter
- **Services tab** - Running services with status, port, and PID information
- **Dependencies tab** - Inbound and outbound network connections to/from this host

6. Alert Management & Remediation

InfraWatch automatically creates alerts when it detects outdated packages, known vulnerabilities, or version mismatches during scanning. The Alerts page provides tools to triage, acknowledge, and remediate these findings.

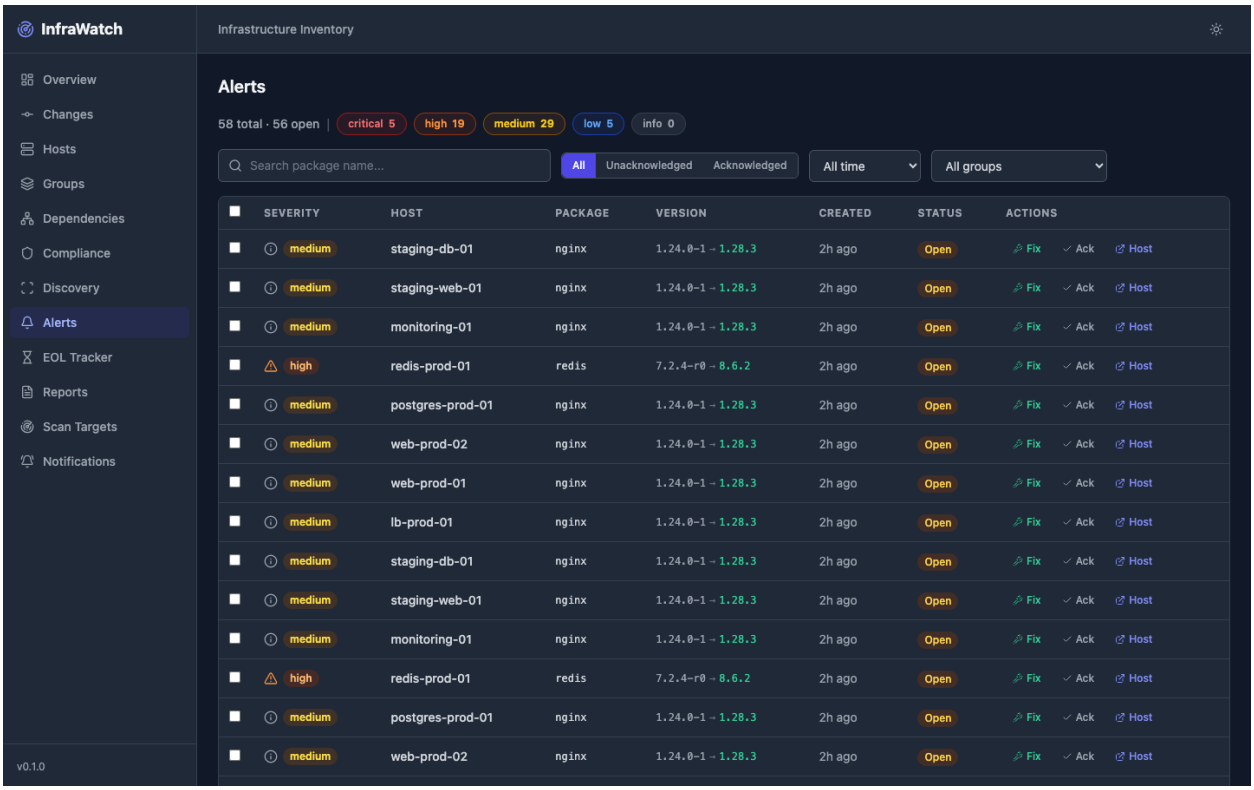

Figure 4: Alert management with severity summary, filtering, and bulk actions

Severity Levels

Level	Color	Description
Critical	Red	Known CVEs or major version gaps requiring immediate attention
High	Orange	Significant version differences or security-relevant updates
Medium	Yellow	Minor version updates available
Low	Blue	Patch-level updates available
Info	Gray	Informational findings, no action required

Alert Actions

- **Acknowledge** - Mark an alert as reviewed (with optional notes). Acknowledged alerts are excluded from notification digests.
- **Bulk Acknowledge** - Select multiple alerts via checkboxes and acknowledge them at once.
- **Remediation (Fix)** - View auto-generated platform-specific commands to resolve the alert. Commands are tailored to the host's package manager (apt, yum, pip, npm, etc.).
- **Bulk Remediation** - Generate a combined remediation plan for multiple selected alerts.

Important: Remediation commands are generated as suggestions. Always review commands before executing them on production systems. Commands may need adjustment for your specific environment.

7. Change Tracking

The Change Feed automatically records every infrastructure change detected during scanning. This provides a complete audit trail of what changed, when, and on which host.

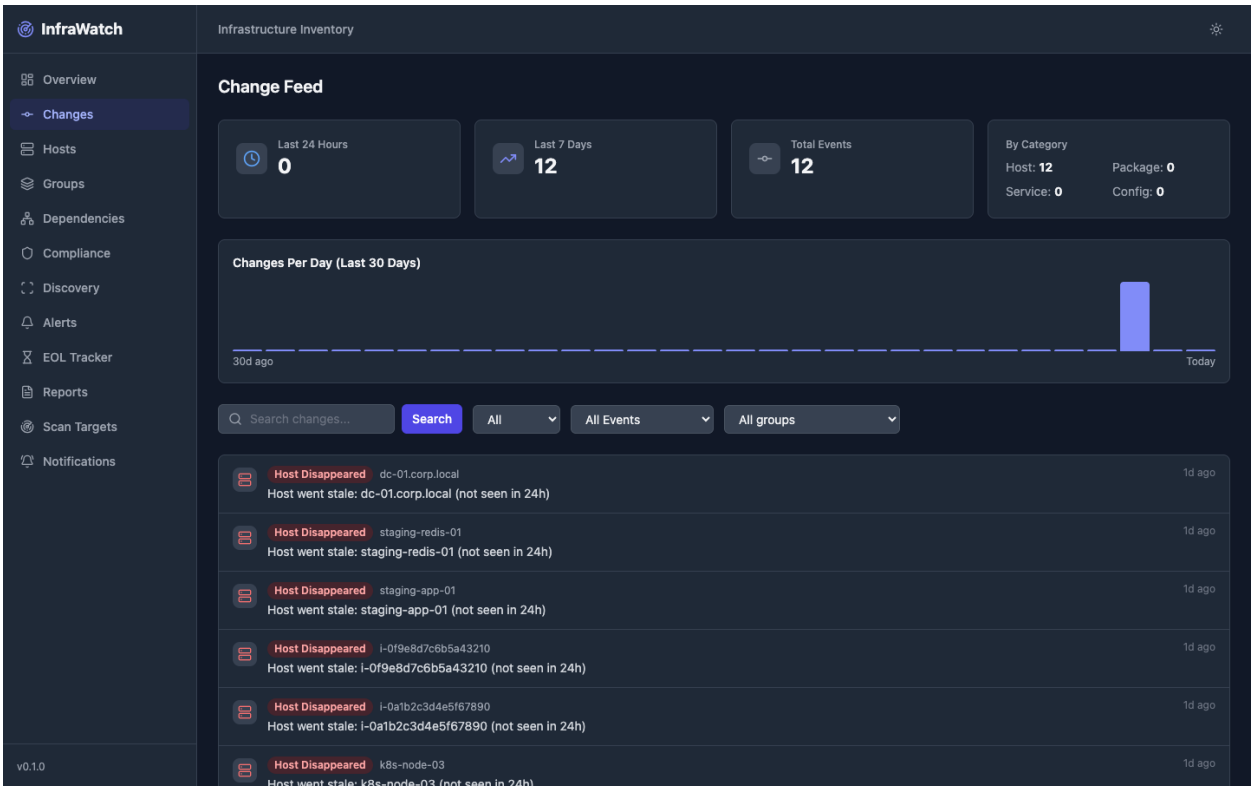

Figure 5: Change Feed with summary cards, 30-day trend chart, and event log

Change Event Types

- **host_discovered** - New host first seen during a scan
- **host_disappeared / host_stale** - Host stopped responding
- **package_added / package_removed / package_updated** - Package inventory changes
- **service_added / service_removed / service_changed** - Service status changes
- **os_changed** - Operating system version change detected
- **ip_changed** - Host IP address changed
- **eol_detected** - Product approaching or past end-of-life

Change Categories

Changes are grouped into four categories for filtering: **host**, **package**, **service**, and **config**. The summary cards show counts for the last 24 hours, last 7 days, and total changes. The 30-day trend chart visualizes

change velocity over time.

8. End-of-Life (EOL) Tracking

The EOL Tracker monitors installed products and operating systems against known end-of-life dates. Running software past its EOL is a security risk, as vendors stop providing patches.

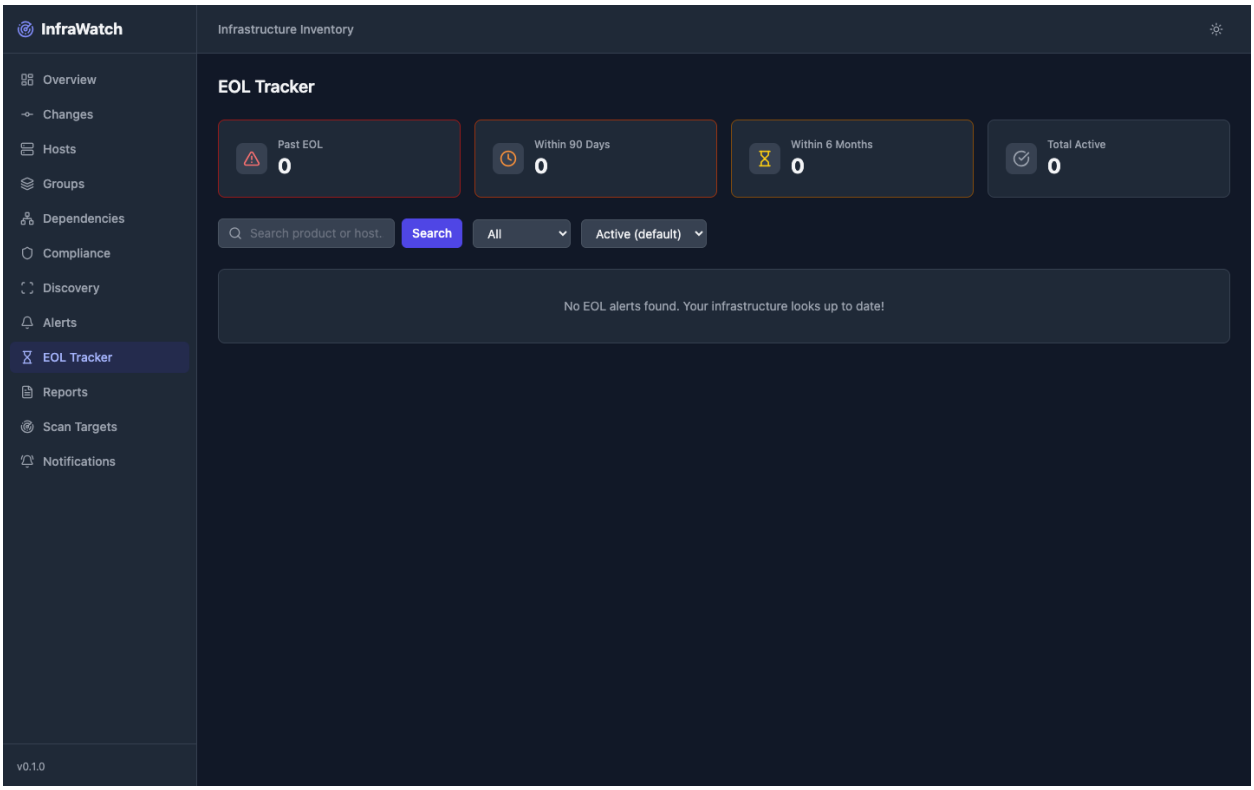

Figure 6: EOL Tracker showing summary cards and alert filters

EOL Alert Statuses

- **Active** - Requires attention; product is approaching or past EOL
- **Acknowledged** - Team is aware; planned for migration
- **Exempted** - Accepted risk with documented reason (e.g., vendor extended support)
- **Resolved** - Product has been updated or replaced

EOL Actions

- **Acknowledge** - Mark as reviewed; stays visible but flagged
- **Exempt** - Exclude from active tracking with a mandatory reason
- **Mark Resolved** - Close the alert after remediation

9. Host Groups

Host Groups let you organize hosts into logical collections such as "Database Servers", "Production Web Servers", or "Staging Environment". Groups are used for filtering across the application and for scoping compliance scores and reports.

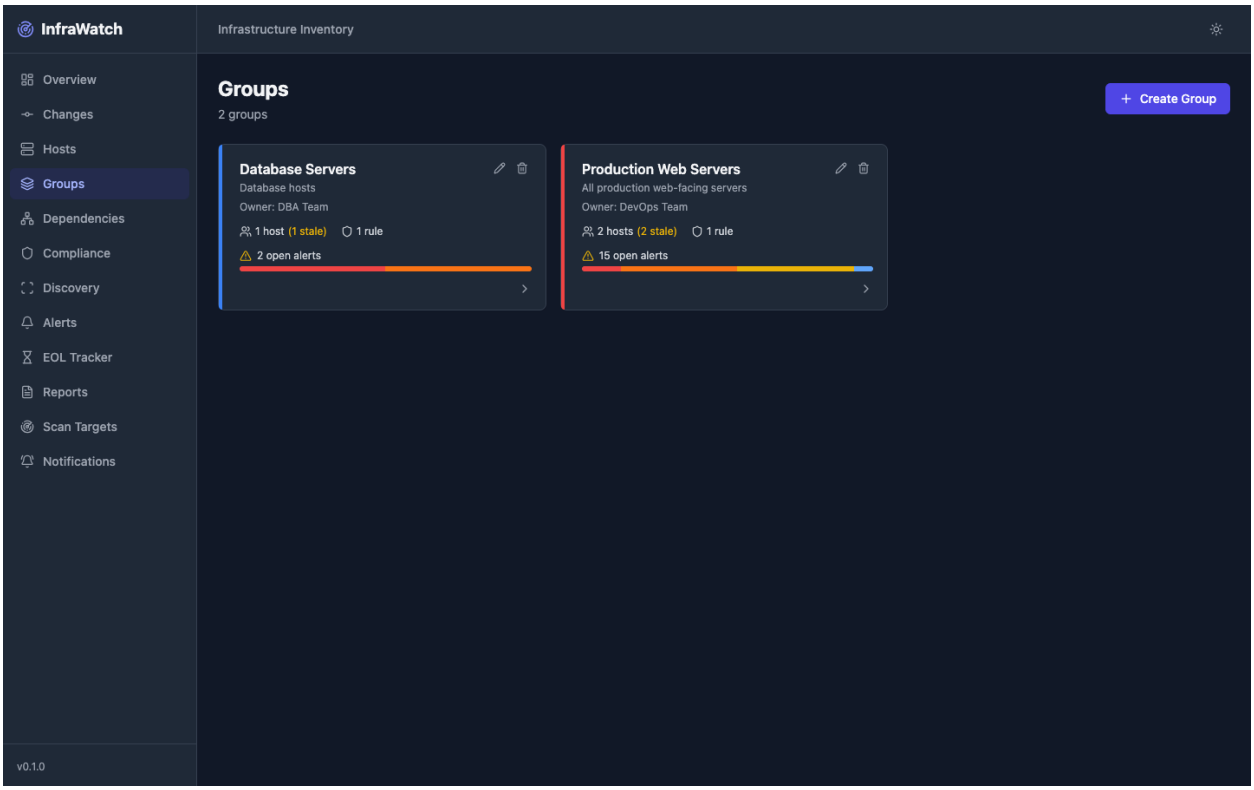

Figure 7: Groups page showing group cards with host count, rules, and alert summary

Creating a Group

- Click **Create Group** in the top-right corner
- Enter a **name**, optional **description**, and optional **owner**
- Choose a **color** for visual identification
- Save and then add hosts manually or via auto-assignment rules

Auto-Assignment Rules

Rules automatically assign newly discovered hosts to groups based on criteria:

- **Hostname pattern** - Regex match on hostname (e.g., `db-.*`)
- **OS match** - Filter by operating system
- **Environment tag** - Match on tagged environment

- **IP range** - Match hosts within a CIDR range

10. Dependency Mapping

The Dependency Map visualizes network connections between hosts in your infrastructure. Connections are automatically discovered during scans by analyzing established TCP connections and listening services.

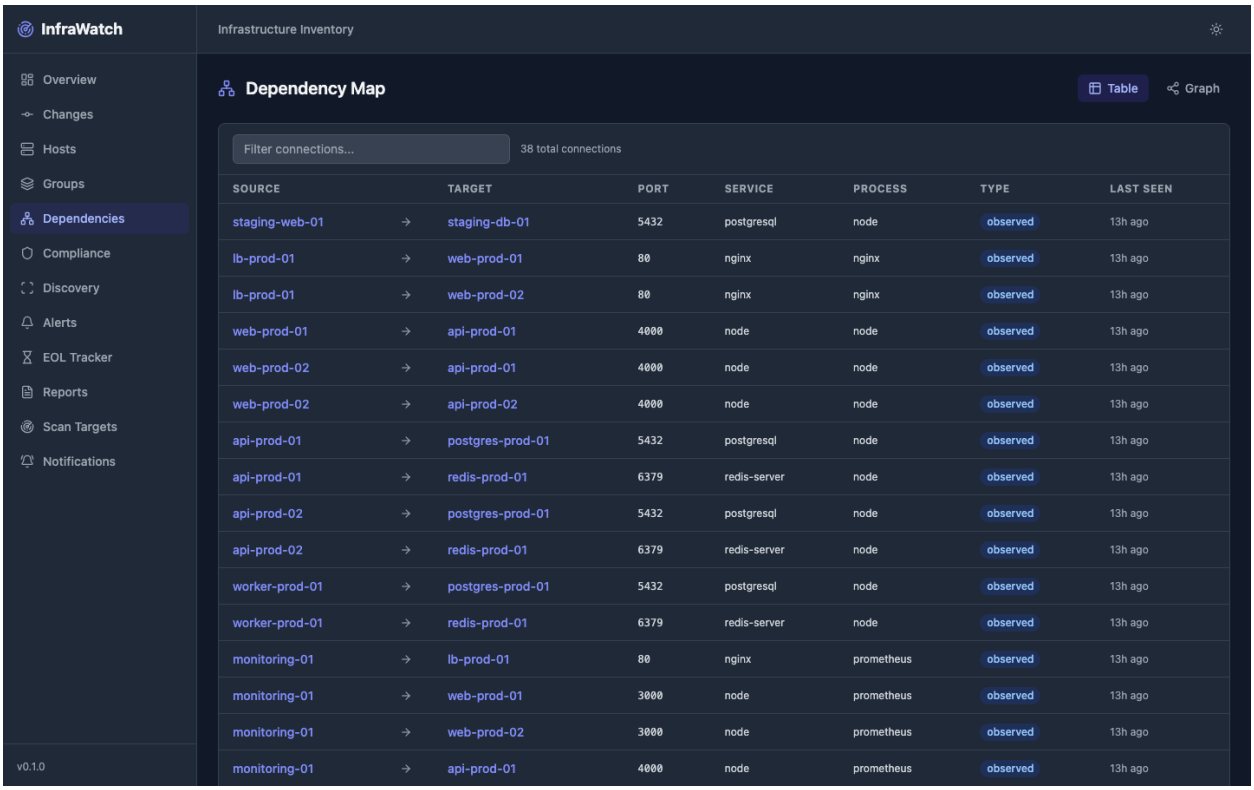

Figure 8: Dependency Map showing source-target connections with protocol and port

View Modes

- **Table View** - Lists all connections with source host, target host, protocol, port, and process information. Supports text filtering.
- **Graph View** - Interactive network graph visualization. Hosts are nodes, connections are edges. Hover over a node to highlight its direct dependencies.

Impact Analysis

The dependency data powers impact analysis: when viewing a host's detail page, the Dependencies tab shows both inbound (services that depend on this host) and outbound (services this host depends on) connections. This helps assess the blast radius of taking a host offline for maintenance.

11. Compliance Scoring

InfraWatch calculates a compliance score (0-100) for each host based on five weighted security factors. Scores are aggregated to group, environment, and fleet levels for executive visibility.

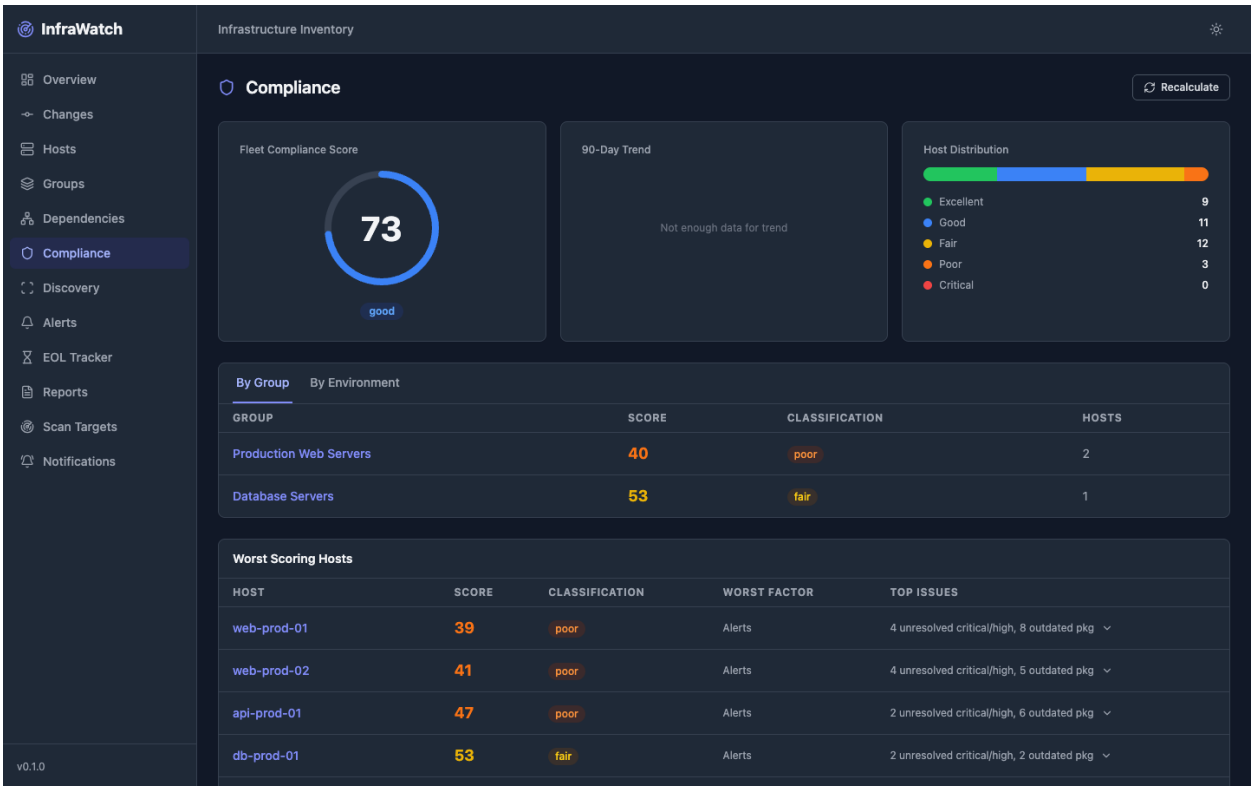

Figure 9: Compliance dashboard with fleet gauge, group/environment scores

Scoring Model

Factor	Weight	Description
Package Currency	35 pts	Ratio of up-to-date packages to total packages
EOL Status	25 pts	No active EOL alerts = full score; past EOL = 0
Alert Resolution	20 pts	Ratio of acknowledged critical/high alerts to total
Scan Freshness	10 pts	Based on time since last successful scan
Service Health	10 pts	Ratio of running services to total configured services

Score Classifications

Classification	Score Range	Action
Excellent	90 - 100	No action needed; maintain current posture
Good	70 - 89	Minor improvements recommended
Fair	50 - 69	Several areas need attention
Poor	30 - 49	Significant remediation required
Critical	0 - 29	Immediate action required

Scores recalculate automatically daily at 2:00 AM and after each scan completes. Click the **Recalculate** button on the Compliance page to trigger an immediate recalculation.

12. Network Discovery

Network Discovery scans your network ranges for live hosts that are not yet in the inventory. Discovered hosts can be promoted to full inventory entries or dismissed.

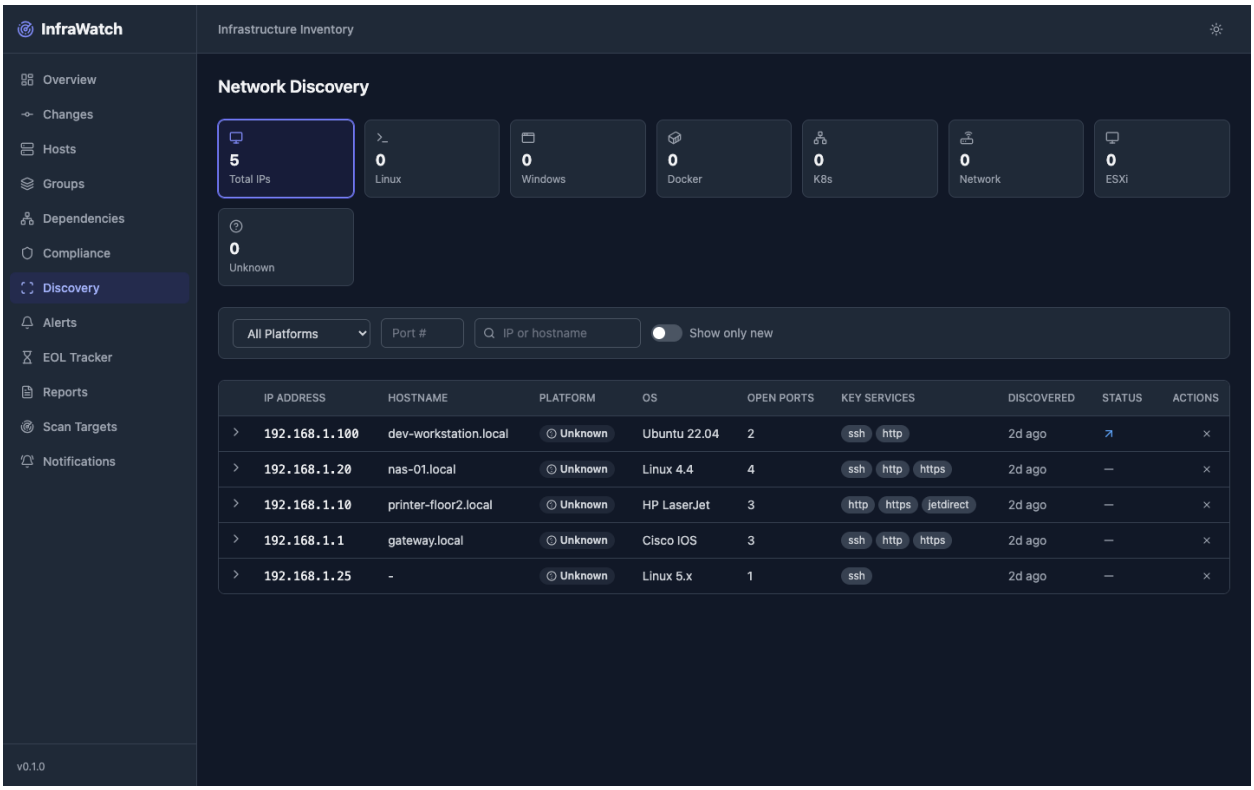

Figure 10: Network Discovery showing discovered IPs with platform breakdown

Discovery Workflow

- Configure a **Network Discovery** scan target with the CIDR ranges to scan
- The scanner uses ARP scanning, ICMP ping, and port scanning (via nmap) to find live hosts
- Discovered hosts appear on this page with their IP, hostname, platform, and open ports
- **Promote** a discovered host to add it to your inventory (it will be assigned to a scan target for ongoing monitoring)
- **Dismiss** a host to hide it from the list (e.g., printers, IoT devices you don't want to track)

Filters

Filter discovered hosts by platform (Linux/Windows/Unknown), port number, IP/hostname search, or whether they were auto-promoted or dismissed.

13. Scan Target Management

Scan Targets define the connection details for reaching your infrastructure. Each target specifies a scanner type, connection configuration, and scan interval.

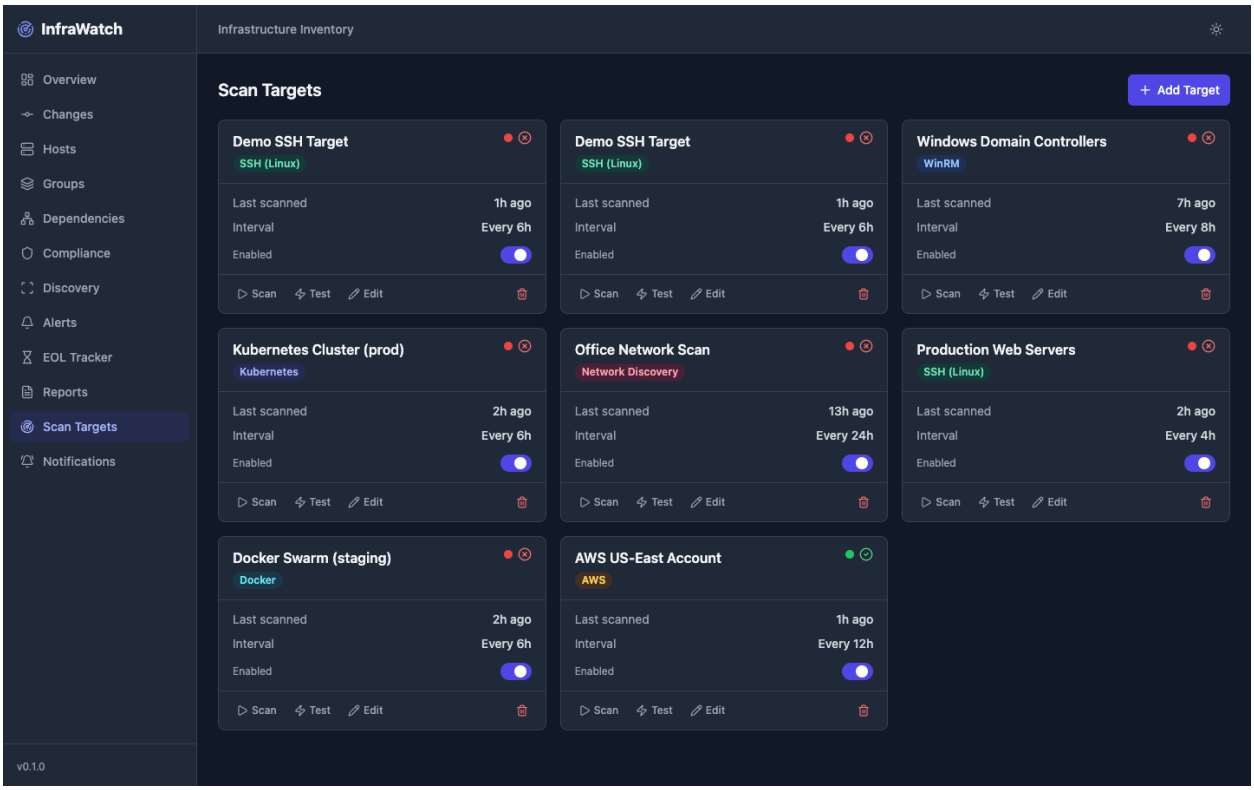

Figure 11: Scan Targets showing configured targets with status and actions

Creating a Scan Target

- Click **Add Target** to open the creation form
- Select the **scanner type** (SSH Linux, WinRM, Kubernetes, AWS, VMware, Docker, or Network Discovery)
- Fill in the **connection configuration** (varies by type - e.g., hostname, port, username, password/key for SSH)
- Set the **scan interval** in hours (how often InfraWatch scans this target)
- Enable or disable the target

Target Actions

- **Scan** - Trigger an immediate scan regardless of the schedule
- **Test** - Test the connection without performing a full scan

- **Edit** - Modify the target's configuration or interval
- **Delete** - Remove the target (does not remove discovered hosts)

Security Note: All connection credentials (passwords, SSH keys, API tokens) are encrypted with AES-256-GCM using the `MASTER_KEY` before storage. They are never exposed through the API or UI.

14. Scheduled Reports

InfraWatch can generate and email PDF reports on a configurable schedule. Reports provide point-in-time snapshots of your infrastructure state.

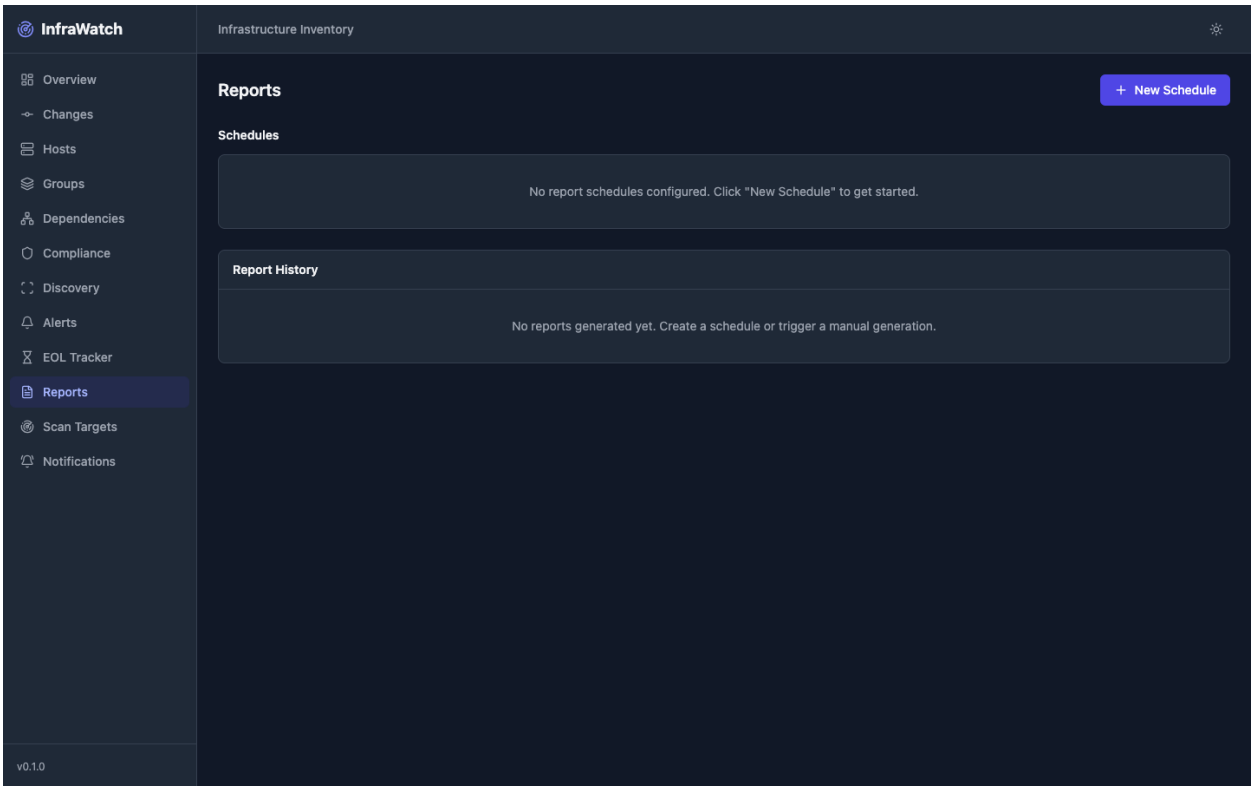

Figure 12: Reports page with schedules and generation history

Report Types

- **Weekly Summary** - Overview of fleet health, new hosts, alerts, and changes
- **EOL Report** - All products approaching or past end-of-life
- **Alert Report** - Open and recently resolved alerts by severity
- **Host Inventory** - Full inventory export with packages and services

Schedule Presets

- **Weekly** - Every Monday at 8:00 AM
- **Bi-weekly** - Every other Monday
- **Monthly** - First day of each month
- **Daily** - Every day at 8:00 AM

- **Custom** - Any cron expression

Reports can also be triggered manually via the **Generate Now** button or previewed as HTML in the browser.

15. Notifications

Configure notification channels to receive real-time alerts about infrastructure events. InfraWatch supports multiple channel types with configurable severity and event filters.

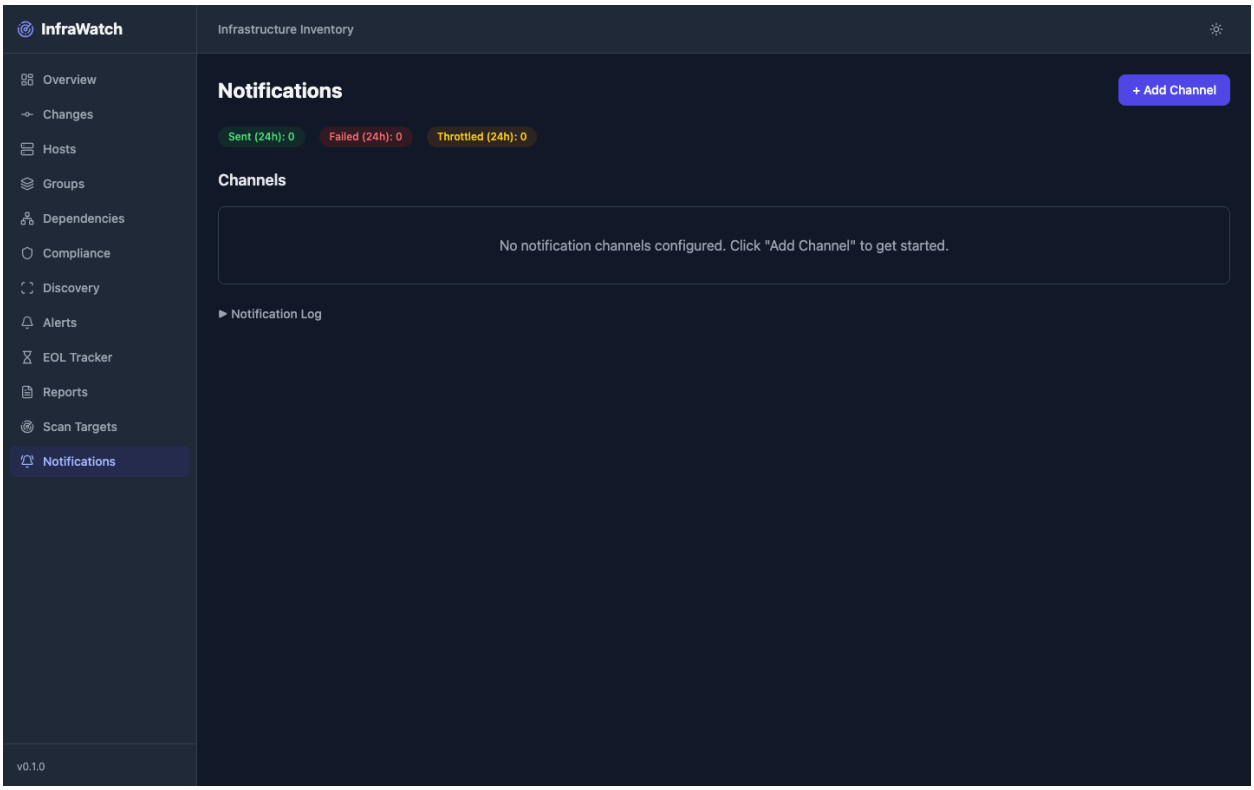

Figure 13: Notifications page showing channel configuration and delivery log

Channel Types

- **Slack** - Posts to a Slack channel via incoming webhook URL
- **Microsoft Teams** - Posts via Teams connector webhook
- **Email** - Sends emails via SMTP (requires SMTP configuration)
- **Generic Webhook** - POSTs JSON to any HTTP endpoint

Event Subscriptions

Each channel can subscribe to specific event types:

- **Alerts** - New vulnerability/version alerts
- **EOL** - End-of-life warnings
- **Host Offline** - Host went stale or unreachable

- **Scan Failures** - Scan target connection failures
- **Daily Digest** - Summary of all activity in the last 24 hours

Use the **Test** button after configuring a channel to verify delivery. The **Notification Log** at the bottom shows recent delivery attempts with success/failure status.

16. API Reference

All functionality is accessible via the REST API at `/api/v1`. Include the `X-API-Key` header in all requests.

Hosts

Method	Endpoint	Description
GET	`/api/v1/hosts`	List hosts (paginated, filterable)
GET	`/api/v1/hosts/:id`	Get host details with packages & services
GET	`/api/v1/hosts/:id/remediation`	Get remediation plan for host

Alerts

Method	Endpoint	Description
GET	`/api/v1/alerts`	List alerts (filterable by severity, host, group)
GET	`/api/v1/alerts/summary`	Alert counts by severity
POST	`/api/v1/alerts/:id/acknowledge`	Acknowledge an alert
POST	`/api/v1/alerts/bulk-acknowledge`	Bulk acknowledge alerts
GET	`/api/v1/alerts/:id/remediation`	Get fix commands for alert

Scan Targets

Method	Endpoint	Description
POST	`/api/v1/targets`	Create scan target
GET	`/api/v1/targets`	List all targets
PATCH	`/api/v1/targets/:id`	Update target
DELETE	`/api/v1/targets/:id`	Delete target
POST	`/api/v1/targets/:id/scan`	Trigger scan
POST	`/api/v1/targets/:id/test`	Test connection

Compliance

Method	Endpoint	Description

GET	`/api/v1/compliance/fleet`	Fleet score, trend, distribution
GET	`/api/v1/compliance/hosts`	Per-host scores (sorted, filtered)
GET	`/api/v1/compliance/groups`	Group-level scores
GET	`/api/v1/compliance/environments`	Environment-level scores
GET	`/api/v1/compliance/trend`	Historical score trend
POST	`/api/v1/compliance/recalculate`	Trigger recalculation

Other Endpoints

Method	Endpoint	Description
GET	`/api/v1/changes`	List change events
GET	`/api/v1/eol/alerts`	List EOL alerts
GET	`/api/v1/groups`	List host groups
GET	`/api/v1/dependencies/connections`	List connections
GET	`/api/v1/dependencies/map`	Full dependency graph
GET	`/api/v1/reports/schedules`	List report schedules
GET	`/api/v1/notifications/channels`	List notification channels
GET	`/api/v1/discovery`	List discovered hosts

17. Updating & Maintenance

Updating InfraWatch

To update to a new version:

```
# Pull the latest code
git pull origin main

# Rebuild and restart containers
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build

# Database migrations run automatically on startup
```

Note: Database migrations are applied automatically when the API server starts. Always back up the database before upgrading to a new version.

Database Backup

```
# Create a backup
docker compose exec postgres pg_dump -U infrawatch infrawatch > backup.sql

# Restore from backup
docker compose exec -T postgres psql -U infrawatch infrawatch < backup.sql
```

Log Access

```
# View API server logs
docker compose logs -f api

# View web server logs
docker compose logs -f web

# View database logs
docker compose logs -f postgres
```

Resource Limits (Production)

The production Docker Compose file enforces these resource limits:

- **PostgreSQL**: 512 MB memory
- **API Server**: 1 GB memory
- **Web (nginx)**: 128 MB memory

Adjust these in `docker-compose.prod.yml` based on your fleet size. Large deployments (500+ hosts) may require increasing the API server memory and database connection pool.

18. Troubleshooting

Web UI shows "Unable to connect" or blank page

- Verify the API server is running: `docker compose ps`
- Check API health: `curl http://localhost:3001/api/v1/health`
- Verify `CORS_ORIGIN` matches the URL you're accessing the UI from
- Check API logs for errors: `docker compose logs api`

Scans fail or time out

- Use the **Test Connection** button on the scan target to verify connectivity
- Ensure the target host allows connections on the required port (SSH: 22, WinRM: 5985)
- For SSH targets, verify the SSH key or password is correct
- For Kubernetes targets, verify the kubeconfig or service account has read access
- Check API logs for detailed error messages

Hosts stuck in 'stale' status

- Verify the scan target is enabled and the interval hasn't elapsed too long
- Trigger a manual scan from the Scan Targets page
- Check if the host is actually reachable on the network
- The StaleHostChecker marks hosts as stale after 24 hours without a successful scan

Compliance scores not updating

- Scores recalculate daily at 2:00 AM and after each scan
- Click **Recalculate** on the Compliance page to force an update
- Check API logs for errors in the ComplianceScorer service

Email notifications not sending

- Verify SMTP settings in the `.env` file
- Use the **Test** button on the notification channel to check delivery
- Check the Notification Log for error messages
- Ensure your SMTP provider allows the sending volume

Database connection errors

- Verify PostgreSQL is running: `docker compose ps postgres`
- Check that `DB_HOST`, `DB_PORT`, and `DB_PASSWORD` are correct
- Ensure the database exists: `docker compose exec postgres psql -U infrawatch -l`
- Check if the connection pool is exhausted (increase `DB_POOL_MAX` if needed)

For additional help or to report bugs, visit the project repository or contact your system administrator.